

Mentor Management System (MMS)

1. Project Overview

❖ System purpose

The Mentor Management System (MMS) is a digital platform designed to help manage and simplify the connection between mentors and mentees. The system allows mentees to search for suitable mentors based on skills, experience, and interests, while mentors can manage their profiles, schedules, and mentoring sessions. It also helps users request mentorship sessions, receive mentor recommendations, and provide feedback after each session.

The backend system acts as the main storage and management center for all platform data, including user profiles, mentor skills, mentee interests, session schedules, feedback records, notifications, and analytics. In addition, the system provides secure RESTful APIs that allow web applications, mobile applications, and other systems to communicate with the platform efficiently. Overall, the Mentor Management System improves the mentoring process by making it more organized, secure, and accessible for mentors, mentees, and administrators.

Business Goals

- **Accessible Mentorship:** Help Cambodian students, fresh graduates, and junior professionals easily find and connect with suitable mentors.
- **Efficient Matching:** Use a skill-based recommendation system to reduce manual searching and improve mentor matching.
- **Session Management:** Provide a clear system for requesting, approving, completing, declining, and cancelling mentorship sessions.
- **Quality Assurance:** Improve platform trust and mentor accountability through ratings and feedback.
- **Scalable Backend Foundation:** Build a secure and well-documented RESTful API that supports future web, mobile, and third-party integrations.
- **Administrative Oversight:** Give administrators tools to manage users, monitor activities, and track platform growth.

❖ Main Problem Solved

- Students and young professionals in Cambodia have difficulty finding suitable mentors in their career or technical field.
- Mentor discovery currently depends on informal platforms such as Facebook, Telegram, LinkedIn, or personal connections.
- There is no organized system for requesting, scheduling, and managing mentorship sessions.
- Existing platforms lack proper feedback and rating systems to measure mentor quality and accountability.

- Administrators and universities cannot easily track mentorship activities, session progress, or platform performance.
- Mentor and mentee profiles are not structured for skill-based matching, making accurate mentor recommendations difficult.

❖ Target Users

The Mentor Management System (MMS) is designed for three main types of users: mentees, mentors, and administrators

- Mentees include students, fresh graduates, and junior professionals who want career guidance and skill development.
- Mentors are experienced professionals who provide guidance, manage sessions, and support mentees.
- Administrators manage users, monitor platform activities, and track system performance through dashboards and reports.

❖ Scope of the Backend System

The backend is a secure, stateless RESTful API with JWT authentication and RBAC for three roles: Admin, Mentor, and Mentee. Key capabilities include:

- **Authentication & Security:** User registration, email verification, and bcrypt password hashing.
- **Mentor Management:** Profile creation, skill cataloging with proficiency levels, and availability control.
- **Mentee Management:** Interest registration and career goal tracking.
- **Skills Catalog:** System-managed catalog with category classification and full CRUD operations.
- **Session Management:** Five-state workflow: Pending, Confirmed, Completed, Cancelled, and No-Show.
- **Recommendation Engine:** Skill-based mentor matching with configurable scoring weights.
- **Feedback & Ratings:** Multi-dimensional rating system with mentor response capability.
- **Notifications & Logging:** In-app notifications for platform events and automatic activity logging.
- **Admin Dashboard:** Statistics, log access, and platform oversight endpoints.
- **Search & Filtering:** Pagination, filtering, sorting, and full-text search across all major endpoints.

Out of scope for MVP: Email delivery, video integration, payment processing, calendar sync, and mobile push notifications to be added in future iterations without modifying the core API contract.

2. Functional Requirements

❖ Authentication & Authorization

- **JWT Authentication:** All protected endpoints secured via Bearer tokens in the Authorization header.
- **RBAC:** Three roles enforced: Admin, Mentor, and Mentee.
- **Password Security:** Hashed using bcrypt with a minimum of 12 salt rounds.
- **Registration & Login:** Secure flows with appropriate HTTP status codes.
- **Email Verification:** Time-limited tokens required before sensitive operations.
- **Password Reset:** Securely generated, time-limited reset tokens sent via email.
- **Account Lockout:** Triggered after a configurable number of failed login attempts.
- **Token Management:** Access tokens expire in 15 minutes; refresh tokens last 7 days, stored in HTTP-only cookies.

❖ Student Management

- **Profile Management:** Authenticated users can retrieve and update their own profile.
- **User Record Fields:** Email, password hash, full name, role, avatar URL, biography, location, verification status, and active status.
- **Soft Deletion:** All user records are soft-deleted to preserve historical data integrity.
- **Role Assignment:** Each user has one role (Admin, Mentor, or Mentee), assigned at registration and immutable unless changed by an Admin.
- **Admin Controls:** Admins can list, search, activate, deactivate, and soft-delete any user account.

❖ Mentor Management

- **Mentor Profile:** Mentors can create and update a dedicated profile separate from the base user record.
- **Skill Associations:** Mentors can link multiple skills from the catalog, each recording proficiency level (Beginner, Intermediate, Expert) and years of experience.
- **Availability Status:** Four states managed: Available, Busy, and Unavailable.
- **Average Rating:** Automatically recalculated after each new feedback submission.
- **Session Counter:** Total sessions automatically incremented upon each successful completion.
- **Profile Completeness:** A completeness percentage is calculated and stored for each mentor profile.

❖ Mentee Management

- **Mentee Profile:** Mentees can create and update a dedicated profile separate from the base user record.
- **Learning Interests:** Mentees can associate multiple skills from the catalog, each with a configurable priority ranking to influence recommendations.
- **Skill Level & Career Goal:** Stores the current skill level (Beginner, Intermediate, Advanced) and career goal text.
- **Session Counter:** Total sessions automatically incremented upon each successful completion.
- **Profile Completeness:** A completeness percentage is calculated and stored for each mentee profile.

❖ Skill and interest management

- **Skill Catalog CRUD:** Full Create, Read, Update, and Delete operations restricted to Admins only.
- **Skill Categories:** Four categories: Technical, Soft Skills, Business, and Creative.
- **Mentor & Mentee Associations:** Mentors link skills and mentees link interests from the same catalog.
- **Uniqueness Constraint:** Duplicate skill or interest associations per profile are not allowed.
- **Inactive Skills:** Excluded from new associations but existing links are preserved for historical integrity.

❖ Session Scheduling

- **Session Request:** Mentees submit requests with mentor ID, topic, description, scheduled date/time, duration, and an optional message.
- **Mentor Approval:** Only the assigned mentor can approve or decline a request; approval requires a meeting link or location.
- **State Machine Workflow:**
 - ✓ Pending → Confirmed (*mentor approval*)
 - ✓ Pending → Declined (*mentor decline*)
 - ✓ Confirmed → Completed (*mentor or mentee marks complete*)
 - ✓ Confirmed → Cancelled (*either party cancels*)
 - ✓ Confirmed → No-Show (*no-show declaration*)
- **Invalid Transitions:** Return HTTP 400 or HTTP 403 as appropriate.
- **Access Control:** Only the assigned mentor, requesting mentee, or an Admin may interact with a session record.

❖ Matching Algorithm

- **Recommendation Engine:** Returns personalized mentor suggestions for authenticated mentees.
- **Scoring Formula:** $\text{Match Score} = (\text{Skill Overlap Ratio} \times 0.50) + (\text{Normalized Rating} \times 0.30) + (\text{Availability Score} \times 0.20)$
 - ✓ **Skill Overlap Ratio:** Matching skills / Total mentee interests
 - ✓ **Normalized Rating:** Average rating / 5.0
 - ✓ **Availability Score:** 1.0 (Available), 0.5 (Busy), 0.0 (Unavailable)
- **Sorted Results:** Recommendations returned in descending order of match score.
- **Filtering Support:** Filter by skill, minimum rating, availability status, and location.
- **Response Object:** Each recommendation includes match score, matching skills, average rating, total sessions, and availability status.

❖ Feedback and Rating

- **Feedback Eligibility:** Mentees can only submit feedback for Completed sessions; non-completed sessions return HTTP 400.
- **Rating Dimensions:** Three independent ratings: Knowledge, Communication, and Helpfulness, each scored 1–5.
- **Overall Rating:** Automatically calculated as the arithmetic mean of the three dimensions.
- **One Feedback Per Session:** Duplicate submissions return HTTP 409 Conflict.
- **Mentor Response:** Mentors can submit exactly one textual response to their received feedback.
- **Visibility Control:** Admins can toggle feedback visibility; default is set to true.
- **Rating Recalculation:** Mentor's aggregate average rating is automatically updated upon each new feedback submission.

❖ Notification System

- **Auto-Generated Notifications:** Triggered for the following events:
 - Session request received (*notifies mentor*)
 - Session approved (*notifies mentee*)
 - Session declined with reason (*notifies mentee*)
 - Session cancellation (*notifies opposing party*)
 - Session reminder 24 hours before scheduled time (*notifies both parties*)
 - Feedback request after session completion (*notifies mentee*)
 - Feedback received (*notifies mentor*)
- **Read/Unread Status:** Each notification record maintains a read/unread state.
- **Retrieval** – Authenticated users can fetch their notification list with pagination and view unread count.
- **Mark as Read** – Individual notifications can be marked as read via a PATCH endpoint.
- **MVP Scope** – Limited to in-app database-stored notifications; email delivery deferred to future phases.

❖ Activity Logging

- **Auto Activity Logging** – Automatically recorded for: user registration, profile updates, skill/interest additions, session requests, approvals, declines, completions, cancellations, feedback submissions, and visibility toggles.
- **Log Entry Fields** – Captures user ID, action type, entity type, entity ID, metadata payload (JSON), IP address, and creation timestamp.
- **Immutable Records** – Log entries cannot be updated or deleted after creation.
- **Admin Only Access** – Activity logs are restricted exclusively to the Admin role.
- **Pagination & Filtering:** Supports filtering by user, entity type, action, and date range with pagination.

❖ Admin Dashboard Statistics

- **Dashboard Statistics** – A dedicated admin endpoint returning: total users, mentors, mentees, sessions, session counts by status, completion rate, and average platform rating.
- **Live Aggregations** – All statistics computed from live database queries at request time.
- **Admin List Access** – Paginated and filterable lists for users, sessions, skills, feedback, and activity logs.

- **Future Export Support** – Architecture supports future CSV or PDF report export endpoints.

❖ Search, Filtering, and Pagination

- **Pagination** – All list endpoints support cursor-based or offset-based pagination, defaulting to 10 items per page.
- **Full-Text Search** – Mentor lists support search by name, biography, skill name, and location.
- **Filtering** – Supports filtering by skill category, minimum rating, availability status, session status, and date range.
- **Sorting Options** – Rating, creation date, total sessions, and match score (descending).
- **Pagination Metadata** – All responses include total items, total pages, and current page.

3. Non-Functional Requirements

❖ Security

- **HTTPS Encryption** – All production data transmission encrypted using TLS 1.2 minimum.
- **Password Hashing** – bcrypt with a minimum of 12 salt rounds in production.
- **Input Validation** – All inputs validated and sanitized against SQL Injection, XSS, and CSRF attacks.
- **Token Expiry** – Access tokens expire in 15 minutes; refresh tokens expire in 7 days, stored in HTTP-only, SameSite=Strict cookies.
- **Authorization Guards** – All endpoints enforce access control ensuring users only access permitted resources.
- **Rate Limiting** – 100 requests/minute globally per IP; 10 requests/minute on sensitive endpoints (login, registration).
- **Account Lockout** – Triggered after 5 consecutive failed login attempts within a 15-minute window.
- **Environment Variables** – All sensitive configuration values (JWT secrets, database credentials) are injected via environment variables, never committed to version control.
- **Error Handling** – Production responses never expose stack traces, database errors, or internal system identifiers.

❖ Performance and Scalability

Response Time Targets:

- Standard GET endpoints — < 150ms
- Session creation/update (POST/PATCH) — < 500ms
- Mentor recommendation engine — < 2,000ms (*up to 1,000 active mentors*)
- Authentication endpoints — < 300ms (*including bcrypt verification*)
- Admin dashboard statistics — < 1,000ms (*live database aggregation*)
- **Stateless Architecture** – No server-side session state, enabling horizontal scaling behind a load balancer.
- **Database Indexes** – Applied on frequently queried columns: **average_rating**, **skill_id**, **mentor_id**, **availability_status**, and **created_at**.
- **Connection Pooling** – Configured via TypeORM with min 5 and max 20 connections.
- **Pagination Enforced** – All large list queries are paginated to prevent unbounded database queries

❖ Maintainability and Code Quality

- **Strict TypeScript** – Enforced via strict: true in tsconfig.json; no any types in production code.
- **Clean Architecture** – Strict separation between controllers, services, and repositories.
 - ✓ **Controllers** – Handle HTTP request/response only; no business logic.
 - ✓ **Services** – Contain all business rules and orchestration logic.
 - ✓ **Repositories** – Handle all database interactions; not called directly from controllers.
- **Code Quality** – ESLint and Prettier enforced for consistent code style.
- **API Documentation** – All public endpoints documented via Swagger/OpenAPI decorators with auto-generated interactive docs.
- **Database Migrations** – All schema changes managed through TypeORM migrations; direct production schema modifications are prohibited.

❖ Error Handling and Validation

- **Global Exception Filter** – Catches all unhandled exceptions and transforms them into a standard error response format.
- **DTO Validation** – All request bodies are validated at the controller boundary using class-validator DTOs before reaching the service layer.
- **Validation Errors** – Return HTTP 400 with a structured response listing all validation errors.
- **Business Rule Violations** – Return appropriate 4xx status codes with descriptive messages.
- **Production Safety** – Stack traces are never exposed in client-facing responses in production.

❖ API Response Standards

Refer to Section 9 for the complete API response standard specification, including success, paginated, and error response schema definitions

4. Suggested Backend Architecture

❖ Layered Architecture Design

The **Mentor Management System** backend adopts a Clean Architecture pattern enforced through NestJS module conventions, ensuring strict separation of concerns, testability, and swappable infrastructure without modifying business logic.

- **Presentation Layer** – Handles HTTP requests and responses only; no business logic permitted. (*Controllers*)
- **Guard / Middleware Layer** – Manages JWT authentication, RBAC authorization, rate limiting, and request logging. (*Guards & Middleware*)
- **DTO Layer** – Defines and validates request/response data shapes at the boundary. (*class-validator decorators*)
- **Business Logic Layer** – Encapsulates all business rules, orchestration, and calculations. (*Services*)
- **Data Access Layer** – Handles all database queries and persistence operations. (*TypeORM Repositories*)
- **Domain Layer** – Represents database tables and defines entity relationships. (*TypeORM Entities*)

❖ Dependency Injection

The **MMS** backend is built on **Node.js** using an **Object-Oriented Programming (OOP)** approach, leveraging NestJS's built-in IoC (Inversion of Control) container to manage all component dependencies. Classes and decorators define the structure, where services are injected into controllers, repositories are injected into services, and all bindings are declared through module providers arrays. This enforces the Dependency Inversion Principle and facilitates unit testing through mock injection.

❖ Recommended Folder Structure

```
src/
├─ config/                # Application configuration (DB, JWT, Rate Limit)
├─ constants/             # Enums, string constants, role definitions
├─ core/
│   ├─ decorators/        # Custom decorators (e.g., @CurrentUser)
│   ├─ guards/            # JwtAuthGuard, RolesGuard, EmailVerifiedGuard
│   ├─ filters/           # GlobalExceptionHandler
│   ├─ interceptors/      # LoggingInterceptor, TransformInterceptor
│   └─ middlewares/       # RateLimitMiddleware, RequestLoggerMiddleware
├─ database/
│   ├─ migrations/        # TypeORM migration files
│   └─ seeds/             # Seed data for development and testing
├─ modules/
│   ├─ auth/              # AuthController, AuthService, AuthModule
│   ├─ user/              # UserController, UserService, UserModule
│   ├─ mentor/            # MentorController, MentorService, MentorModule
│   ├─ mentee/            # MenteeController, MenteeService, MenteeModule
│   ├─ skill/             # SkillController, SkillService, SkillModule
│   ├─ session/           # SessionController, SessionService, SessionModule
│   ├─ matching/          # MatchingController, MatchingService, MatchingModule
│   ├─ feedback/          # FeedbackController, FeedbackService, FeedbackModule
│   ├─ notification/      # NotificationController, NotificationService
│   ├─ admin/             # AdminController, AdminService, AdminModule
│   └─ activity-log/      # ActivityLogService, ActivityLogModule
├─ common/
│   ├─ dto/               # Shared DTOs (PaginationDto, ResponseDto)
│   └─ utils/             # Utility functions (hash, token generation)
├─ app.module.ts
└─ main.ts
```


5. Technology Stack

Layer Component	Technology Selected	Architectural Justification
Core runtime Engine	Node.js	Non-blocking, event-driven I/O runtime ideal for high-concurrency RESTful API workloads. LTS stability guarantees suitability for production deployment.
Programming Language	OOP of TypeScript	Strict static typing eliminates an entire class of runtime type errors. Decorator support aligns with NestJS's metadata-driven architecture. Improves code readability and maintainability in large codebases.
Backend Framework	NestJS	Enterprise-grade modular architecture with built-in dependency injection, decorators, guards, interceptors, and pipes. Enforces Clean Architecture conventions and reduces boilerplate significantly compared to Express.
Relational Database	MySQL	The MMS backend uses MySQL as its relational database, providing ACID compliance and strong support for complex JOINS, indexed queries, and structured data storage. It is well-suited for managing multi-entity mentorship data with defined relationships across users, sessions, skills, and feedback records.
ORM	TypeORM	First-class TypeScript support with decorator-based entity definitions. Supports migrations, relationships, repository pattern, and query builder. Integrates natively with Nest.js.
Input Validation	class-validator & class transformer	Declarative DTO-based validation using decorators. Enables automatic transformation of plain request objects into typed class instances. Integrates with NestJS's global validation pipe.
Authentication	JSON Web Tokens (JWT)	Stateless authentication mechanism suitable for horizontal scaling. Widely supported across frontend and mobile clients. Compliant with RFC 7519
Password Hashing	bcrypt	Adaptive cryptographic hash function with configurable work factor. Industry-standard choice for secure password storage, resistant to brute-force and rainbow table attacks.
Logging	Pino	Extremely fast structured JSON logger with minimal performance overhead. Native NestJS integration available. Outputs machine-readable logs suitable for aggregation pipelines.
Testing Framework	Jest + Supertes	Jest provides comprehensive unit and integration testing capabilities with built-in mocking, code coverage, and TypeScript support. Supertest enables HTTP-level end-to-end API testing without a running server.
API Documentation	Swagger / OpenAPI	Auto-generated, interactive API documentation from Nest.js decorators. Provides a standardized API specification for frontend integration, QA testing, and academic review.
Containerization	Docker + Docker Compose	Ensures environmental consistency across development, staging, and production. Eliminates "works on my machine" issues and simplifies deployment pipeline configuration
Reverse Proxy	Reverse Proxy	High-performance HTTP server for SSL termination, request routing, rate limiting, and serving static assets in production environments

6. Database Design

❖ Entity Relationship Summary

- A User may have at most one Mentor profile (1:0..1 relationship).
- A User may have at most one Mentee profile (1:0..1 relationship).
- A Mentor may have many MentorSkills (1:N); each Skill may appear in many MentorSkills (M:N resolved through junction table).
- A Mentee may have many MenteeInterests (1:N); each Skill may appear in many MenteeInterests (M:N resolved through junction table).
- A Session references exactly one Mentor and one Mentee.
- A Session may have at most one Feedback record (1:0..1 relationship).
- A Session may have at most one SessionRequest record.
- A User may have many Notifications (1:N).
- A User may generate many ActivityLogs (1:N)

❖ Table Definitions

Table: users

Column Name	Data Type	Constraints	Description
id	VARCHAR (100)	PRIMARY KEY, NOT NULL	Universally unique user identifier generated by the database.
email	VARCHAR(100)	UNIQUE, NOT NULL, NDEXED	User email address used for authentication and communication.
password_hash	VARCHAR(255)	NOT NULL	Plain text passwords must never be stored.
full_name	VARCHAR(100)	NOT NULL	User's full display name
role	VARCHAR(20)	NOT NULL, CHECK IN ('ADMIN','MENTOR','MENTEE')	User role determines access level and available features
avatar_url	VARCHAR(255)	NULLABLE	URL to the user's profile image stored in object storage
bio	TEXT	NULLABLE	Short user biography or professional summary
location	VARCHAR(100)	NULLABLE	User's city or province, used in search and recommendations
is_email_verified	BOOLEAN	DEFAULT false, NOT NULL	Indicates whether the user has verified their email address
is_active	BOOLEAN	DEFAULT true, NOT NULL	Account active status. Inactive accounts cannot authenticate.
failed_login_count	INTEGER	DEFAULT 0	Counter for consecutive failed login attempts for lockout logic.
locked_until	TIMESTAMP	NULLABLE	Account lockout expiry timestamp. NULL if not locked
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Record creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Last record update timestamp, managed by TypeORM
deleted_at	TIMESTAMP	NULLABLE	Soft deletion timestamp. NULL indicates an active record

Table: Mentors

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique mentor profile identifier
user_id	UUID	FK → users.id, UNIQUE, NOT NULL	Reference to the associated user account. One-to-one relationship enforced by UNIQUE constraint
years_of_experience	INTEGER	NULLABLE, CHECK ≥ 0	Total years of professional experience in the mentor's primary domain
total_sessions	INTEGER	DEFAULT 0, NOT NULL	Cumulative count of sessions conducted. Incremented on session completion.
average_rating	DECIMAL(3,2)	DEFAULT 0.00, NOT NULL	Aggregate average rating computed from all feedback records. Updated on each new feedback submission.
availability_status	VARCHAR(20)	DEFAULT 'AVAILABLE', CHECK IN ('AVAILABLE','BUSY','UNAVAILABLE')	Current availability status influencing matching score calculation.
profile_completeness	INTEGER	DEFAULT 0, CHECK BETWEEN 0 AND 100	Percentage score reflecting the completeness of the mentor profile
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Profile creation timestamp.
updated_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Last profile update timestamp.

Table: mentee

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique mentor profile identifier
user_id	UUID	FK → users.id, UNIQUE, NOT NULL	Reference to the associated user account.
Current_level	VARCHAR(50)	NULLABLE, CHECK IN ('BEGINNER','INTERMEDIATE','ADVANCED')	Mentee's current self-assessed skill level.
career_goal	TEXT	NULLABLE	Mentee's stated career objective or professional aspiration.
total_sessions	INTEGER	DEFAULT 0, NOT NULL	Cumulative count of sessions attended. Incremented on session completion.
profile_completeness	INTEGER	DEFAULT 0, CHECK BETWEEN 0 AND 100	Current availability status influencing matching score calculation.
created_at	INTEGER	DEFAULT NOW(), NOT NULL	Profile creation timestamp.
updated_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Last profile update timestamp.

Table: Skills

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique mentor profile identifier
name	VARCHAR(100)	UNIQUE, NOT NULL, INDEXED	Canonical skill name. Must be unique across the catalog.
category	VARCHAR(50)	NULLABLE, CHECK IN ('BEGINNER','INTERMEDIATE','ADVANCED')	Skill classification category for browsing and filtering
is_active	BOOLEAN	DEFAULT true, NOT NULL	Active status. Inactive skills are hidden from new associations but preserved for historical records.
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Skill creation timestamp

Table: mentor_skills

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
mentor_id	UUID	FK → mentors.id, NOT NULL, INDEXED	Reference to the mentor profile.
Skill_id	UUID	FK → skills.id, NOT NULL, INDEXED	Reference to the skill in the catalog.
proficiency_level	VARCHAR(20)	NOT NULL, CHECK IN ('BEGINNER','INTERMEDIATE','ADVANCED','EXPERT')	Mentor's self-assessed proficiency level for this skill.
years_using	INTEGER	NULLABLE, CHECK ≥ 0	Years of practical experience applying this specific skill.
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Association creation timestamp.

Table: mentee_interests

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
mentee_id	UUID	FK → mentors.id, NOT NULL, INDEXED	Reference to the mentor profile.
Skill_id	UUID	FK → skills.id, NOT NULL, INDEXED	Reference to the skill in the catalog.
priority	INTEGER	NOT NULL, CHECK ≥ 1	Priority ranking (lower number = higher priority) influencing recommendation weight.
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Association creation timestamp.

Table: session

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
mentee_id	UUID	FK → mentors.id, NOT NULL, INDEXED	Reference to the mentor profile.
mentor_id	UUID	FK → mentors.id, NOT NULL, INDEXED	Reference to the session's assigned mentor.
topic	VARCHAR(200)	NOT NULL	Descriptive title or subject of the session.
description	TEXT	NULLABLE	Detailed session agenda or objectives provided by the mentee
scheduled_at	TIMESTAMP WITH TIME ZONE	NOT NULL, INDEXED	Agreed session start date and time in UTC.
duration_minutes	INTEGER	DEFAULT 60, NOT NULL, CHECK > 0	Agreed session duration in minutes
meeting_link	VARCHAR(255)	NULLABLE	Video meeting URL or physical meeting location set upon approval.
status	VARCHAR(20)	DEFAULT 'PENDING', NOT NULL, INDEXED	Current session status within the defined state machine.
cancellation_reason	TEXT	NULLABLE	Reason provided when a session is cancelled or declined
completed_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Timestamp when the session was marked as completed.
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Session request creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Last session record update timestamp.

Table: session_requests

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
session_id	UUID	FK → sessions.id, UNIQUE, NOT NULL	Reference to the associated session. One-to-one with sessions.
message	TEXT	NULLABLE	Optional introductory message from the mentee to the mentor
request_status	VARCHAR(20)	DEFAULT 'PENDING', NOT NULL	Status of the request itself: PENDING, APPROVED, DECLINED.
responded_at	TIMESTAMP	NULLABLE	Timestamp when the mentor responded to the request.
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Association creation timestamp.

Table: feedback

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
session_id	UUID	FK → sessions.id, UNIQUE, NOT NULL	Reference to the associated session. One-to-one with sessions.
mentee_id	UUID	FK → mentees.id, NOT NULL	Reference to the mentee submitting the feedback.
mentor_id	UUID	K → mentors.id, NOT NULL, INDEXED	Reference to the mentor receiving the feedback. Used for rating aggregation.
rating_knowledge	INTEGER	NOT NULL, CHECK BETWEEN 1 AND 5	Mentee's rating of the mentor's domain knowledge and expertise.
Rating_communication	INTERGER	NOT NULL, CHECK BETWEEN 1 AND 5	Mentee's rating of the mentor's communication clarity and style.
Rating_helpfulness	INTERGER	NOT NULL< CHECK BETWEEN 1 AND 5	Mentee's rating of how helpful the session was toward their goals.
Overall_rating	DECIMA (3,2)	NOT NULL	System-calculated arithmetic mean of the three dimensions
Comment	TEXT	NULLBLE	Optional free-text qualitative feedback from the mentee.
Mentor_response	TEXT	NULLBLE	Optional mentor response to the feedback. May be submitted once.
Is_visible	BOLLEAN	DEFAULT true, NOT NULL	Feedback submission timestamp
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Association creation timestamp.
Update_at	TIMESTAMP	DEFAULT NOE(), NOT NULL	Last feedback record update timestamp.

Table: notification

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
User_id	VARCHAR(50)	NOT NULL	Reference to the recipient user.
type	UUID	FK → mentees.id, NOT NULL	Notification event type (e.g., SESSION_REQUESTED, SESSION_APPROVED).
title	VARCHAR(200)	NOT NULL	Short notification title displayed in the notification list.
message	TEXT	NOT NULL	Full notification message body.
entity_type	VARCHAR(50)	NULLABLE	Related entity type (e.g., SESSION, FEEDBACK)

entity_id	UUID	NULLABLE	Related entity UUID for deep linking within the client application
is_read	BOOLEAN	DEFAULT false, NOT NULL, INDEXED	Read status. Used to compute unread notification count
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL	Association creation timestamp.

Table: activity_logs

Column Name	Data type	Constraints	Description
id	UUID	PRIMARY KEY, NOT NULL	Unique junction record identifier
user_id	UUID	FK → users.id, NULLABLE, INDEXED	Reference to the user who performed the action. NULLABLE for system generated events.
action	VARCHAR(100)	NOT NULL, INDEXED	Action label (e.g., USER_REGISTERED, SESSION_APPROVED, FEEDBACK_SUBMITTED).
entity_type	VARCHAR(50)	NOT NULL	Type of entity affected by the action (e.g., USER, SESSION, FEEDBACK).
entity_id	UUID	NULLABLE	Flexible JSON payload for storing action-specific contextual data.
metadata	JSONB	NULLABLE	Association creation timestamp.
ip_address	VARCHAR(45)	NULLABLE	Client IP address captured from the HTTP request (supports IPv4 and IPv6)
created_at	TIMESTAMP	DEFAULT NOW(), NOT NULL, INDEXED	Immutable log entry creation timestamp. Records are never updated.

7. Use case specification

UC-001: Register User

Actor: Unregistered User (Public)

Preconditions: The email address submitted must not already exist in the users table.

Main Flow:

1. User submits a POST request to `/api/v1/auth/register` with email, password, full name, and role.
2. System validates the request body using the RegisterUserDto class-validator schema.
3. System queries the users table to confirm the email address does not already exist.
4. System hashes the submitted password using bcrypt with the configured salt rounds.
5. System creates a new user record with the hashed password and the submitted profile fields.
6. System creates a corresponding mentor or mentee profile record based on the user's registered role.
7. System generates a time-limited email verification token and persists it.

8. System dispatches an email verification notification (MVP: in-app notification; future: email delivery).
9. System records a USER_REGISTERED entry in the activity_logs table.
10. System returns HTTP 201 Created with the user's ID, email, role, and email verification status.

Alternative Flow:

If the email already exists, system returns HTTP 409 Conflict:

"Email address is already registered."

If the request body fails validation, system returns HTTP 400 Bad Request with validation error details.

Postconditions: A new user account and associated profile are persisted. An email verification notification is generated.

UC-002: Login User

Actor: Registered User

Preconditions: User must have a registered account. Account must not be locked.

Main Flow:

1. User submits POST request to `/api/v1/auth/login` with email and password.
2. System retrieves the user record by email.
3. System verifies the submitted password against the stored password hash using bcrypt comparison.
4. System resets the failed_login_count to zero upon successful verification.
5. System generates a short-lived JWT access token (15 minutes) and a long-lived refresh token (7 days).
6. System stores the refresh token in an HTTP-only cookie.
7. System returns HTTP 200 OK with the access token, user ID, role, and full name.

Alternative Flow:

_ If the email is not found, return HTTP 401 Unauthorized: "Invalid credentials."

_ If the password does not match, increment failed_login_count and return HTTP 401.

_ If failed_login_count reaches the threshold (default: 5), set locked_until for 15 minutes and return HTTP 423 Locked.

_ If the account is locked, return HTTP 423 Locked with remaining lockout time.

Postconditions: User receives a valid JWT access token. Refresh token is stored securely in an HTTP-only cookie.

UC-003: Mentor Adds Skill

Actor: Authenticated Mentor

Preconditions: User must be authenticated as MENTOR role. The specified skill must exist and be active in the skills catalog.

Main Flow:

1. Mentor submits POST request to `/api/v1/mentors/me/skills` with skillId, proficiencyLevel, and optional yearsUsing.
2. System extracts and validates the authenticated mentor identity from the JWT.
3. System verifies that the specified skill ID exists in the active skills catalog.

4. System checks that the mentor does not already have an association with this skill.
5. System creates a new mentor_skills junction record.
6. System recalculates and updates the mentor's profile_completeness score.
7. System records a MENTOR_SKILL_ADDED entry in the activity_logs table.
8. System returns HTTP 201 Created with the created mentor skill record.

Alternative Flow:

_ If skill ID does not exist or is inactive, return HTTP 404 Not Found.

_ If the mentor already has this skill, return HTTP 409 Conflict: "Skill already added to mentor profile."

Postconditions: The skill association is persisted. Mentor profile completeness is Recalculated.

UC-004: Mentee Adds Interest

Actor: Authenticated Mentee

Preconditions: User must be authenticated as MENTEE role. The specified skill must exist and be active.

Main Flow:

1. Mentee submits POST request to `/api/v1/mentees/me/interests` with priority.
2. System validates the authenticated mentee's identity from the JWT.
3. System verifies the skill exists in the active catalog.
4. System checks for duplicate interest association.
5. System creates the mentee_interests junction record with the supplied priority value.
6. System recalculates and updates the mentee's profile_completeness score.
7. System returns HTTP 201 Created with the created interest record.

Alternative Flow:

_ If skill does not exist, return HTTP 404 Not Found.

_ If interest already exists for the mentee, return HTTP 409 Conflict.

Postconditions: Interest is persisted. Mentee profile completeness is updated. The recommendation engine will incorporate this interest in the next recommendation request.

UC-005: Mentee Requests Session

Actor: Authenticated Mentee (email verified)

Preconditions: Mentee must be authenticated, email-verified, and the target mentor must exist and be active.

Main Flow:

1. Mentee submits POST request to `/api/v1/sessions/request` with mentor ID, topic, scheduled time, duration, and optional message.
2. System validates the request body using CreateSessionRequestDto.
3. System verifies the email-verified guard is satisfied.
4. System confirms the mentor ID references an existing, active mentor.
5. System creates a new session record with status PENDING.
6. System creates an associated session_requests record with the mentee's message.
7. System generates a SESSION_REQUESTED in-app notification for the mentor.
8. System records a SESSION_REQUESTED entry in activity_logs.

9. System returns HTTP 201 Created with the session record.

Alternative Flow:

- _ If mentor does not exist, return HTTP 404 Not Found.
- _ If email is not verified, return HTTP 403 Forbidden.
- _ If the scheduled time is in the past, return HTTP 400 Bad Request.

Postconditions: Session request is persisted with PENDING status. Mentor receives in-app notification.

UC-006: Mentor Approves Session Request

Actor: Authenticated Mentor

Preconditions: The session must exist, be in PENDING status, and belong to the authenticated mentor.

Main Flow:

1. Mentor submits PATCH request to `/api/v1/sessions/:id/approve` with meetingLink.
2. System retrieves the session by ID and verifies it belongs to the authenticated mentor.
3. System validates the current session status is PENDING.
4. System updates the session status to CONFIRMED and sets the meeting link.
5. System updates the session_requests record status to APPROVED and sets responded_at.
6. System generates a SESSION_APPROVED in-app notification for the mentee.
7. System records a SESSION_APPROVED entry in activity_logs.
8. System returns HTTP 200 OK with the updated session record.

Alternative Flow:

- _ If the session does not belong to the authenticated mentor, return HTTP 403 Forbidden.
- _ If the session is not in PENDING status, return HTTP 400 Bad Request: invalid state transition.

Postconditions: Session status is CONFIRMED. Mentee receives approval notification with meeting link.

UC-007: Mentor Declines Session Request

Actor: Authenticated Mentor

Preconditions: The session must exist, be in PENDING status, and belong to the authenticated mentor.

Main Flow:

1. Mentor submits PATCH request to `/api/v1/sessions/:id/decline` with an optional reason.
2. System verifies session ownership and PENDING status.
3. System updates the session status to DECLINED and stores the cancellation reason.
4. System updates the session_requests record to DECLINED with responded_at timestamp.
5. System generates a SESSION_DECLINED notification for the mentee with the provided reason.
6. System records a SESSION_DECLINED entry in activity_logs.
7. System returns HTTP 200 OK with the updated session record.

Alternative Flow: If the session does not belong to an authenticated mentor, return HTTP 403 Forbidden.

Postconditions: Session is declined. Mentee is notified of the decline reason.

UC-008: System Recommends Mentors

Actor: Authenticated Mentee

Preconditions: Mentee must be authenticated and must have at least one registered learning interest.

Main Flow:

1. Mentee submits GET request to `/api/v1/matching/recommended-mentors` with optional query filters.
2. System retrieves the mentee's registered interests from the `mentee_interests` table.
3. System queries mentors whose `mentor_skills` overlap with the mentee's interests.
4. System calculates a match score for each candidate mentor using the weighted formula: Skill Match (50%) + Rating (30%) + Availability (20%).
5. System applies any additional filters supplied in the query parameters.
6. System sorts results by match score in descending order.
7. System applies pagination and returns the result set with match metadata.

Alternative Flow:

_ If mentee has no registered interests, return HTTP 400 Bad Request: "Please add at least one learning interest to receive recommendations."

_ If no matching mentors are found, return HTTP 200 OK with an empty data array.

Postconditions: Mentee receives a personalized, scored list of recommended mentors.

UC-009: Mentee Submits Feedback

Actor: Authenticated Mentee

Preconditions: The referenced session must exist, be in COMPLETED status, belong to the authenticated mentee, and must not have existing feedback.

Main Flow:

1. Mentee submits POST request to `/api/v1/feedback` with session ID, three rating dimensions, and optional comment.
2. System validates the request body using `CreateFeedbackDto`.
3. System retrieves the session and verifies its status is COMPLETED.
4. System confirms the mentee is the requesting party of the referenced session.
5. System checks no feedback record already exists for this session.
6. System calculates the `overall_rating` as the arithmetic mean of the three dimension ratings.
7. System creates the feedback record.
8. System recalculates and updates the mentor's aggregate `average_rating`.
9. System generates a FEEDBACK_RECEIVED in-app notification for the mentor.
10. System records a FEEDBACK_SUBMITTED entry in `activity_logs`.
11. System returns HTTP 201 Created with the feedback record.

Alternative Flow:

- _ If session status is not COMPLETED, return HTTP 400 Bad Request.
- _ If feedback already exists for the session, return HTTP 409 Conflict.
- _ If the mentee is not the session's mentee, return HTTP 403 Forbidden.

Postconditions: Feedback is persisted. Mentor average rating is updated. Mentor is notified of new feedback.

UC_010: Admin Views Dashboard

Actor: Authenticated Administrator

Preconditions: User must be authenticated with the ADMIN role.

Main Flow:

1. Admin submits GET request to `/api/v1/admin/dashboard`
2. System verifies the authenticated user holds the ADMIN role via RolesGuard.
3. System executes aggregation queries against users, mentors, mentees, sessions, and feedback tables.
4. System computes total user count, total mentor count, total mentee count, total session count, session counts by status, session completion rate, and average platform rating.
5. System returns HTTP 200 OK with the complete statistics payload.

Alternative Flow: If the user is not ADMIN role, return HTTP 403 Forbidden.

Postconditions: Administrator receives current platform statistics for monitoring and reporting.

8. API Endpoint Specifications

❖ Authentication Module

+ Post api/auth/register

- _ Description: register a new mentor or mentee user account
- _ Auth required: None (Public)
- _ Role allowed: Public
- _ Success response: HTTP 201 created

Request Body:

```
{  
  "email": "mentor@example.com",  
  "password": "SecurePassword123!",  
  "fullName": "John Doe",  
  "role": "MENTOR",  
}
```

```
"bio": "Senior Software Engineer with 7 years of experience.",
"location": "Phnom Penh, Cambodia"
}
```

Success Response (201):

```
{
"success": true,
"message": "Registration successful. Please verify your email.",
"data": {
"userld": "550e8400-e29b-41d4-a716-446655440000",
"email": "mentor@example.com",
"role": "MENTOR",
"isEmailVerified": false
}
}
```

Error Responses:

- _ 400 Bad Request: Validation failed (invalid email format, weak password, missing required fields).
- _ 409 conflict: Email address is already registered.

+ Post api/auth/login

- _ Description: Authenticate user and receive JWT access token and refresh token
- _ Auth required: None (Public)
- _ Role allowed: Public
- _ Success response: HTTP 200 ok

Request Body:

```
{
"email": "mentor@example.com",
"password": "SecurePassword123!"
}
```

Success Response (200):

```
{
"success": true,
"message": "Login successful.",
"data": {
"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"userld": "550e8400-e29b-41d4-a716-446655440000",
"fullName": "John Doe",
"role": "MENTOR",
"isEmailVerified": true}
}
```

Error Responses:

- _ 401 Unauthorized: Invalid email or password.
- _ 423 Locked: Account locked due to excessive failed login attempts.

❖ User Module

+ GET api/auth/me

- _ Description: Retrieve the authenticated user's own profile.
- _ Auth required: Bearer JWT
- _ Role allowed: ADMIN, MENTOR, MENTEE
- _ Success response: HTTP 200 ok

Success Response (200):

```
{
  "success": true,
  "message": "User profile fetched successfully.",
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "mentor@example.com",
    "fullName": "John Doe",
    "role": "MENTOR",
    "avatarUrl": null,
    "bio": "Senior Software Engineer.",
    "location": "Phnom Penh, Cambodia",
    "isEmailVerified": true,
    "createdAt": "2026-04-01T08:00:00.000Z"
  }
}
```

Error Responses:

- _ 401 Unauthorized: Missing or invalid JWT access token.

+ PATCH api/auth/me

- _ Description: Update the authenticated user's profile fields.
- _ Auth required: Bearer JWT
- _ Role allowed: ADMIN, MENTOR, MENTEE
- _ Success response: HTTP 200 ok

Request body (all fields optional):

```
{
  "fullName": "John Smith",
  "bio": "Updated biography text.",
  "location": "Siem Reap, Cambodia",
  "avatarUrl": "https://storage.example.com/avatars/uuid.jpg"
}
```

Error Responses:

- _ 400 Bad request: Validation failed on submitted fields.
- _ 401 Unauthorized: Missing or invalid JWT access token.

❖ Mentor Module

+ GET api/auth/mentors

- _ Description: Retrieve paginated list of mentors with optional filters
- _ Auth required: Bearer JWT
- _ Role allowed: ADMIN, MENTEE
- _ Success response: HTTP 200 OK (Paginated)

Query Parameters:

- _ page (default), limit (default: 10)
- _ search: Full text search by name, bio, or location
- _ skill: Filter by skill name.
- _ minRating: Minimum average rating (0.0 – 5.0)
- _ availability: Filter by AVAILABLE, BUSY, UNAVAILABLE>
- _ sortBy: Options rating, totalSessions, createdAt.

Error Responses:

- _ 401 Unauthorized: Missing or invalid JWT.

+ Post api/auth/skills

- _ Description: Associate a skill from the catalog with the authenticated mentor's profile.
- _ Auth required: Bearer JWT
- _ Role allowed: MENTOR
- _ Success response: HTTP 201 Created

Request Body:

```
{
  "skillId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "proficiencyLevel": "EXPERT",
  "yearsUsing": 5
}
```

Error Responses:

- _ 403 Forbidden — User is not a MENTOR.
- _ 404 Not Found — Skill ID does not exist.
- _ 409 Conflict — Skill already added to mentor profile.

+ PATCH api/auth/availability

- _ Description: Update the authenticated mentor's availability status.
- _ Auth required: Bearer JWT
- _ Role allowed: MENTOR
- _ Success response: HTTP 200 OK

Request Body:

```
{
  "availabilityStatus": "BUSY"
}
```

Error Responses:

- _ 400 Bad Request — Invalid availability status value.
- _ 403 Forbidden — User is not a MENTOR.

❖ Mentee Module

+ POST api/auth/interests

- _ Description: Associate a learning interest skill with the authenticated mentee's profile.
- _ Auth required: Bearer JWT
- _ Role allowed: MENTEE
- _ Success response: HTTP 201 Created

Request Body:

```
{
  "skillId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "priority": 1
}
```

Error Responses:

- _ 403 Forbidden: User is not a MENTEE.
- _ 404 Not Found: Skill ID does not exist.
- _ 409 Conflict: Interest already added to mentee profile.

9. API Response Standard

All API responses from the MentorKhet backend must conform to the following standardized JSON response schemas. These formats must be enforced through a global TransformInterceptor and a global AllExceptionsFilter implemented in NestJS.

- **Standard Success Response**

Used for all successful non-paginated responses (HTTP 200, HTTP 201, HTTP 204).

```
{
  "success": true,
  "message": "Action completed successfully.",
  "data": {}
}
```

- **Standard Paginated Response**

Used for all list-returning endpoints that implement pagination. The meta object is mandatory.

```
{
  "success": true,
  "message": "Data fetched successfully.",
  "data": [],
  "meta": {
    "totalItems": 100,
    "itemCount": 10,
    "itemsPerPage": 10,
  }
}
```



```
"totalPages": 10,  
"currentPage": 1  
}  
}
```

- **Standard Error Response**

Used for all error responses (HTTP 4xx and HTTP 5xx). Stack trace must be excluded in production.

```
{  
"success": false,  
"statusCode": 400,  
"error": "Bad Request Exception",  
"message": "Validation failed: email must be a valid email address; password must be at least 8 characters.",  
"timestamp": "2026-05-19T14:05:00.000Z",  
"path": "/api/v1/sessions/request"  
}
```

10. Authentication and Security design

- **JWT Tokens:** Access tokens expire in 15 minutes (stored in client memory); refresh tokens expire in 7 days (stored in HTTP-only, SameSite=Strict cookies). Access token payload carries: sub, email, role, and iat, signed with HS256.
- **Password Hashing:** All passwords hashed using bcrypt with minimum 12 salt rounds. Plain text passwords are never stored or logged. bcrypt compare is used for constant-time verification.
- **Role-Based Access Control (RBAC):** RolesGuard applied globally via @Roles() decorator. Insufficient role returns HTTP 403.
 - ✓ Admin – Full access to all endpoints, users, skills, sessions, feedback, and logs.
 - ✓ Mentor – Manages own profile, skills, availability, sessions, and feedback responses.
 - ✓ Mentee – Manages own profile, browses mentors, requests sessions, and submits feedback.
- **Email Verification Guard:** EmailVerifiedGuard blocks access to sensitive endpoints if is_email_verified is false, returning HTTP 403.
- **Rate Limiting:** 100 requests/minute globally; 10 requests/minute on /auth/register and /auth/login.
- **Account Lockout Policy:** Triggered after 5 failed attempts within 15 minutes; returns HTTP 423 until lockout expires.
- **Data Privacy and Access Isolation:** Users can only access their own resources; all queries enforce WHERE userId = authenticatedUserId.
- **Additional Security:** HTTPS with TLS 1.2+, helmet security headers, TypeORM parameterized queries, and all secrets injected via environment variables.

11. OOP Design Principles

- **Encapsulation:** Entity state is mutated only through internal methods (e.g., `session.approve()`, `session.decline()`, `session.complete()`), preventing invalid state transitions. Feedback rating calculation is encapsulated within `feedback.calculateOverallRating()`.
- **Abstraction:** Services depend on repository interfaces rather than concrete implementations. `MatchingService` and `NotificationService` hide internal logic behind clean interfaces, keeping controllers unaware of implementation details.
- **Inheritance:** A shared `BaseEntity` abstract class provides common `id`, `createdAt`, and `updatedAt` fields extended by all domain entities. Domain-specific exceptions like `SessionOwnershipException` extend NestJS's `HttpException`.
- **Polymorphism:** A `ScoringStrategy` interface allows multiple scoring implementations (e.g., `SkillBasedScoringStrategy`, `RatingWeightedScoringStrategy`) to be injected at runtime, enabling algorithm swapping without modifying `MatchingService`.
- **Single Responsibility (SRP):** Each class has one responsibility: controllers handle HTTP, services handle business logic, repositories handle database access, guards handle authorization, and interceptors handle cross-cutting concerns.
- **Dependency Inversion (DIP):** High-level services depend on abstractions rather than concrete implementations. NestJS's IoC container resolves bindings at runtime.
- **Interface Segregation (ISP):** Interfaces are narrowly scoped (e.g., `IUserQueryService`, `IUserCommandService`, `IAuthService`) so consuming classes depend only on what they need.

12. Deployment Architecture

- Docker configuration: Multi-stage Dockerfile: first stage compiles TypeScript; second stage copies only compiled output and production `node_modules` for a minimal image.
- Docker Compose Service:
 - ✓ **mms-api:** Node.js 20 / NestJS backend, exposes port 3000 internally.
 - ✓ **MySQL:** MySQL database service with data persisted to a named Docker volume.
 - ✓ **Nginx:** Reverse proxy handling SSL termination, request routing, and rate limiting on ports 80 and 443.
- **Nginx Reverse Proxy:** Acts as the ingress point for all external traffic; handles SSL/TLS termination, HTTP-to-HTTPS redirection, proxies `/api/` requests to port 3000, and manages gzip compression.
- **Environment Variable Configuration:** All secrets (JWT, database credentials) injected via `.env` file in development and via secrets manager or CI/CD pipeline in production. `.env` must never be committed to version control; use `.env.example` for documentation. Example:

```
# Application
NODE_ENV=development
PORT=3000
API_PREFIX=api/v1
# Database
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=mentorkhet_user
DB_PASSWORD=your_db_password
```

```
DB_NAME=mentorkhet_db
DB_SSL=false
# JWT
JWT_ACCESS_SECRET=your_very_long_random_access_secret_here
JWT_REFRESH_SECRET=your_very_long_random_refresh_secret_here
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
# bcrypt
BCRYPT_ROUNDS=12
# Rate Limiting
RATE_LIMIT_TTL=60
RATE_LIMIT_LIMIT=100
# Email (Future)
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USER=noreply@mentorkhet.com
MAIL_PASSWORD=your_smtp_password
```

- **Deployment Checklist:**
 - ✓ Run TypeORM migrations before startup (npm run migration:run)
 - ✓ Verify all environment variables are set and non-empty
 - ✓ Configure Nginx SSL via Let's Encrypt or managed certificate service
 - ✓ Enable Docker health checks targeting GET /api/v1/health
 - ✓ Configure database volume persistence and automated backups
 - ✓ Set Docker resource limits (memory, CPU) on the API container

13. Testing Strategy

- Testing Pyramid
 - ✓ **Unit Testing** (*Jest*): Service methods, utility functions, scoring algorithm, and rating calculations. Target: $\geq 80\%$ statement coverage.
 - ✓ **Integration Testing** (*Jest + TypeORM test database*): Repository queries, database constraints, foreign keys, and transaction behavior. Target: $\geq 70\%$ coverage.
 - ✓ **End-to-End Testing** (*Jest + Supertest*): Full HTTP request-to-response cycle, authentication flows, and role permission enforcement. Target: All critical user flows.
- Unit Testing:
 - ✓ **MatchingService**: Verify scoring formula (Skill 50% + Rating 30% + Availability 20%) with edge cases.
 - ✓ **FeedbackService**: Verify `calculateOverallRating()` arithmetic mean across all valid rating combinations.
 - ✓ **Session State Machine**: Verify all valid and invalid state transitions produce correct outcomes or exceptions.
 - ✓ **AuthService**: Verify token generation, bcrypt comparison, and account logout logic.
- Integration Testing:

- ✓ Unique constraint enforcement on `users.email`, `mentor_skills`, `mentee_interests`, and `feedback.session_id`.
- ✓ Cascading behavior on soft deletion and foreign key constraint enforcement.
- ✓ TypeORM migration correctness and repository filtering, sorting, and pagination.
- End-to-End Testing:
 - ✓ **Full Mentorship Flow** – Register → Login → Add Skills/Interests → Request Session → Approve → Complete → Submit Feedback.
 - ✓ **Authentication Flow** – Register → Login → Access Protected Endpoint → Token Expiry → Refresh Token.
 - ✓ **RBAC Enforcement** – Verify HTTP 403 for unauthorized role actions and HTTP 401 for unauthenticated requests.
 - ✓ **Recommendation Engine** – Verify mentors are correctly ranked by match score.
 - ✓ **Admin Dashboard** – Verify statistics reflect correct aggregate counts after operations.
- **Test Configuration:** Jest configured with `ts-jest`, coverage thresholds: 80% statements/functions/lines, 70% branches.

14. MVP Scope

❖ Must Have (MVP Core)

#	Feature	Acceptance Criteria
1	Authentication and role guard	Register, login, JWT issuance, RBAC enforcement on all protected endpoints.
2	User Profile Management	Authenticated users can retrieve and update their own profile. Soft delete implemented.
3	Mentor Profile	Mentors can create and update profiles. Average rating and total sessions automatically maintained.
4	Mentee Profile	Mentees can create and update profiles. Career goal and current level fields supported.
5	Skills Catalog	Admins can manage skills. Skills categorized and soft-deletable. Active status filter applied.
6	Mentor Skills	Mentors can add/remove skills with proficiency level. Duplicate prevention enforced.
7	Mentee Interests	Mentees can add/remove interests with priority ranking. Duplicate prevention enforced.
8	Matching Recommendation	Recommendation endpoint returns mentors scored by formula (50/30/20) sorted by match score.
9	Session Request and Approval	Full session state machine implemented: PENDING → CONFIRMED → COMPLETED/CANCELLED/NO_SHOW.
10	Feedback and Rating	Three-dimensional feedback with overall rating calculation. One feedback per session enforced. Mentor rating aggregation updated.
11	Admin Dashboard	Dashboard statistics endpoint returns all required aggregated metrics.

❖ Should have (Post-MVP Priority)

These features add significant value and should be targeted for the second development iteration:

- In-app notification system with read/unread status and paginated retrieval.

- Activity logging for all mutation operations with an admin-accessible log endpoint.
- Full-text search, advanced filtering, and sorting across all list endpoints.
- Swagger/OpenAPI interactive documentation accessible at

[/api/docs](#)

- Docker Compose production-ready configuration with Nginx reverse proxy.

Nice to have (future roadmap)

These features represent the platform's growth roadmap beyond the initial project scope

- Transactional email delivery for verification, reminders, and notifications.
- CSV and PDF report export for admin dashboard data.
- Google Calendar integration for automated session scheduling.
- Payment and subscription system for premium mentor access.
- Real-time WebSocket and mobile push notification support via Firebase.
- Advanced analytics dashboards with historical trend data.

15. Final Summary

❖ Project Significance

MMS addresses a structural gap in Cambodia's educational and professional development ecosystem, where the lack of a standardized digital platform for mentor-mentee matching has left career guidance to informal, inequitable channels. MMS provides a structured, algorithm-driven, and accountability-enforced solution that benefits students and young professionals without established networks.

❖ Technical Excellence

- **Architecture** – Clean Architecture with strict separation between HTTP handling, business logic, and data access, producing a maintainable and extensible codebase.
- **Security** – Multi-layered design covering JWT authentication, bcrypt hashing, RBAC, rate limiting, account lockout, and data access isolation.
- **Data Integrity** – Relational schema with constraints, indexes, and soft deletion ensuring consistency and historical record preservation.
- **Scalability** – Stateless API, database indexing, connection pooling, and Docker containerization support horizontal scaling.
- **Code Quality** – Strict TypeScript, ESLint/Prettier, comprehensive test coverage, and Swagger documentation aligned with professional standards.

16.3 Academic Alignment This document satisfies university-level backend system design criteria by covering functional and non-functional requirements, normalized database schema, formal use cases, API specifications, and OOP principles. The technology stack — NestJS, TypeScript, MySQL, TypeORM, JWT, bcrypt, Jest, Docker, and Swagger — represents a coherent, industry-validated choice aligned with scalability and maintainability goals.

16.4 Conclusion MMS is a socially purposeful platform built on sound engineering principles and documented to production-ready standards. The three-week roadmap, MVP prioritization, and phased feature roadmap demonstrate realistic delivery planning alongside long-term growth ambition, providing a complete specification for Engineering, QA, DevOps, and University Evaluators.